

Documentation

Team-8

17/06/2025

1 Team Member

Ismail Hossain

Bodrul Amin Murad

Rayhan Uddin

2 Introduction

This project implements a simple four-bit calculator on the Nexys A7 (Artix-7 XC7A100T) FPGA board. Using the on-board switches as two 4-bit operands (SW0–SW3 for A, SW4–SW7 for B) and push-buttons BTN15 (add) and BTN14 (subtract), the design performs the selected operation in hardware. The 8-bit result is converted to BCD and shown on the board's seven-segment displays, enabling immediate, real-time feedback for inputs.

3 Concept Description

User Interface (Switches)
Sequential control (compute trigger)
Conversion and display multiplexing (binary→BCD → 7-segment)

A simple, low-latency calculator performing addition and subtraction on any two 4-bit numbers. It serves as a hands-on demo of:

- Combinational logic (ALU)
- Sequential control (compute trigger)
- Conversion and display multiplexing (binary→BCD → 7-segment)

Devices and Components:

1. FPGA (Xilinx Artix-7 XC7A100T on Nexys A7): Central processing unit for managing the calculator operations.

2. User Interface

- **DIP Switches (SW0–SW3, SW4–SW7):**
 - SW0–SW3 sample the 4-bit operand A
 - SW4–SW7 sample the 4-bit operand B
- **Push-Buttons (BTN15, BTN14, BTN0):**
 - BTN15 → “Add” operation trigger
 - BTN14 → “Subtract” operation trigger
- **7-Segment Display:** Displays information such as result

Operation Decoder: Small HDL block that detects which push-button is pressed and encodes the corresponding ALU function (add or subtract).

Arithmetic Logic Unit (ALU): Combinational logic module that takes two 4-bit operands plus an operation code and produces an 8-bit binary result.

Binary-to-BCD Converter: HDL implementation of the “double-dabble” or shift-and-add-3 algorithm to translate the 8-bit binary ALU output into two decimal digits for human-readable display.

Power Supply & Decoupling

- **Voltage Regulator (LM7805):** Provides a stable 5V supply to the FPGA and other components.
- **Decoupling Capacitors:** Distributed across the board to smooth supply rails and minimize switching noise during arithmetic and I/O toggling.

4 Project/Team management

Group Member	Task 01	Task 02	Task 03	Task 04
Bodrul Amin Murad	Project Idea	Implementation	FPGA Programming	PCB Design
Rayhan Uddin	Project Idea	Implementation	FPGA Programming	PCB Design
Ismail Hossain	Project Idea	Implementation	FPGA Programming	PCB Design

5 Technologies

- Kicad
- VHDL
- FPGA

6 Implementation

VHDL and FPGA Implementation

Our calculator is built as a hierarchy of synthesizable VHDL modules, each targeting a specific function in the dataflow. At the top level, a single entity interconnects these sub-modules, handling clocking and reset:

1. Operand Registers

- Latch the 4-bit inputs from the board's switches on the rising edge of a "compute" control signal.

2. Operation Decoder

- Monitors the two operation buttons and generates a 1-bit code indicating Add or Subtract.

3. Arithmetic Logic Unit (ALU)

- Performs the combinational addition or subtraction on the registered operands, producing an 8-bit binary result each clock cycle.

4. Binary-to-BCD Converter (Bin2BCD)

- Implements the classic shift-and-add-3 (“double-dabble”) algorithm over a small finite state machine to translate the 8-bit binary into two 4-bit decimal digits.

5. Display Driver (Disp_Driver)

- Time-multiplexes the seven-segment, driving segment lines A–G to present the BCD digits as a stable decimal number.

To ensure correctness before hardware deployment, we performed multi-level simulation:

- **Module-Level Simulation**

- **ALU:** Exhaustive tests over all 4-bit operand pairs for both add/subtract modes, confirming bit-accurate results.
- **Bin2BCD:** Verified against a wide range of binary values to ensure proper decimal digit output.

- **Top-Level Functional Simulation**

Stimulated switch and button inputs in sequence, observed the multiplexed segment outputs, and checked that the displayed decimal result matched expectations for each operation.

After programming the Nexys A7, we manually exercised all combinations of 4-bit inputs (0–15) and both operations. Each result appeared correctly on the seven-segment displays within one “compute” button press. Overflow and underflow cases wrapped naturally in unsigned arithmetic, confirming robust operation across the full input domain.

7 PCB Design

We used Kicad for our PCB design, which has done using following method:

Component Selection: We carefully selected all necessary components for the calculator, including the FPGA, voltage regulator, capacitors, LEDs, switches, and connectors. Each component was chosen based on its electrical characteristics, compatibility, and ability to meet the design requirements.

Schematic Entry: Using KiCad's schematic editor, we created the schematic by placing and connecting all components according to the design. This involved drawing connections between components using wires to create a logical representation of the circuit. We ensured that each connection was correctly made to reflect the actual circuit design, allowing for accurate simulation and layout in subsequent steps.

Annotation and ERC: We annotated the schematic to assign unique identifiers to each component. This step ensures that each component can be individually identified and referenced throughout the design process. We then ran an Electrical Rules Check (ERC) to verify that there were no errors or missing connections in the schematic. The ERC checks for issues such as unconnected pins, missing connections, and incorrect electrical properties, ensuring that the schematic is error-free before proceeding to the layout stage.

Footprint Assignment: We assigned appropriate physical footprints to all components in the schematic. This step ensures that each component will have the correct size and shape on the PCB layout. Footprints represent the physical layout of the component pins and pads on the PCB, and assigning the correct footprints is crucial for accurate component placement and soldering. We used KiCad's extensive library of component footprints, selecting those that matched our components' specifications.

PCB Layout

Import Netlist: We imported the netlist generated from the schematic into the PCB layout tool. The netlist contains information about all the connections that need to be made between components, serving as a blueprint for routing the PCB. Importing the netlist ensures that all connections defined in the schematic are transferred accurately to the PCB layout, providing a clear guide for placing and connecting components.

Component Placement: We arranged components on the PCB, optimizing for trace lengths and ease of routing. Critical components like the FPGA were placed centrally to minimize the length of critical signal paths. We grouped related components

together to simplify routing and reduce signal interference. Proper placement of components is crucial for achieving a compact and efficient layout, reducing the likelihood of errors during manufacturing and assembly.

Routing: We routed the connections between components using the PCB layout editor. Starting with critical traces such as power and ground, we used wide traces to handle higher currents and ensure stable power distribution. We then routed signal traces, ensuring that they were as short and direct as possible to minimize signal delay and interference. Routing involves drawing physical paths for electrical connections on the PCB, which must be done carefully to avoid issues such as crosstalk and electromagnetic interference.

Design Rule Check (DRC): We performed a DRC to ensure the layout met all manufacturing requirements. The DRC checks for issues such as trace width, spacing, and overlap, verifying that the layout adheres to the specified design rules. We fixed any issues identified by the DRC, ensuring that the PCB layout was compliant with manufacturing standards and ready for fabrication. This step is critical for preventing errors that could lead to defects in the final product.

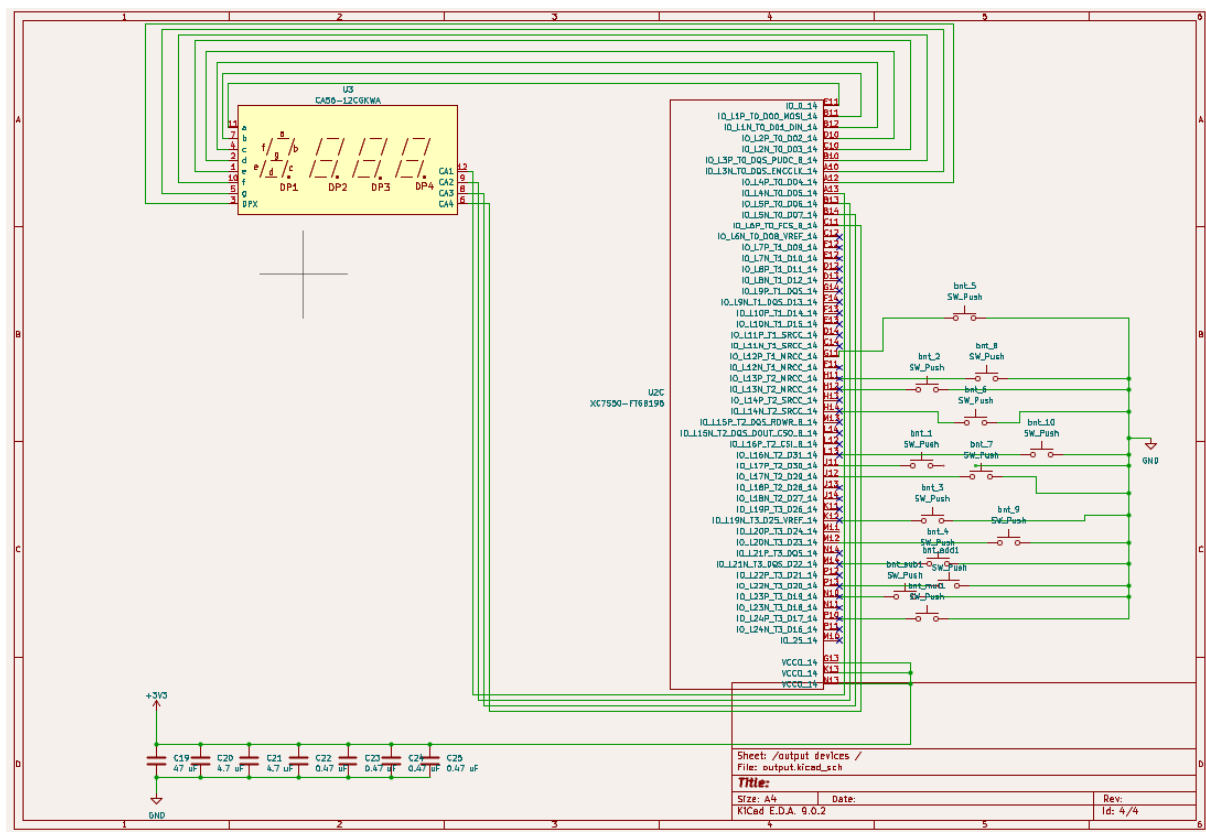


Fig 1: Output Schematic

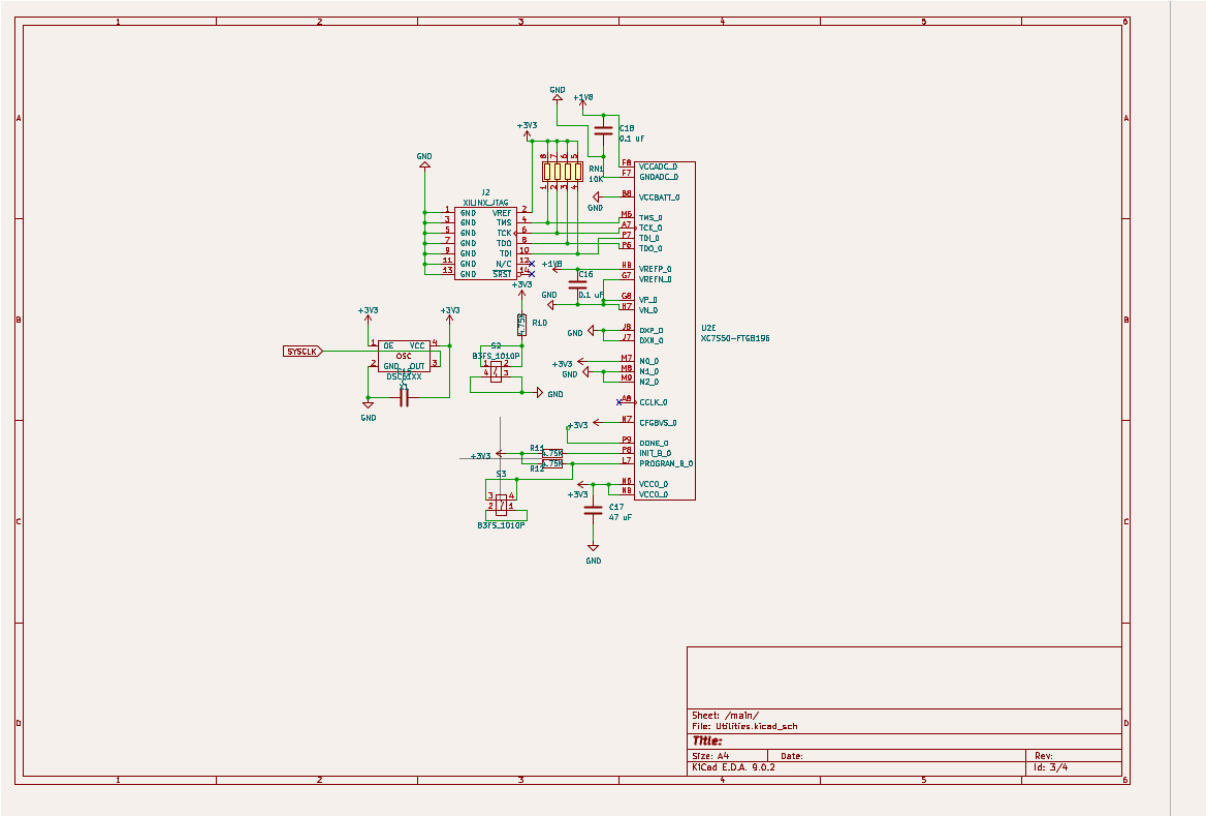


Fig 2: Main Schematic

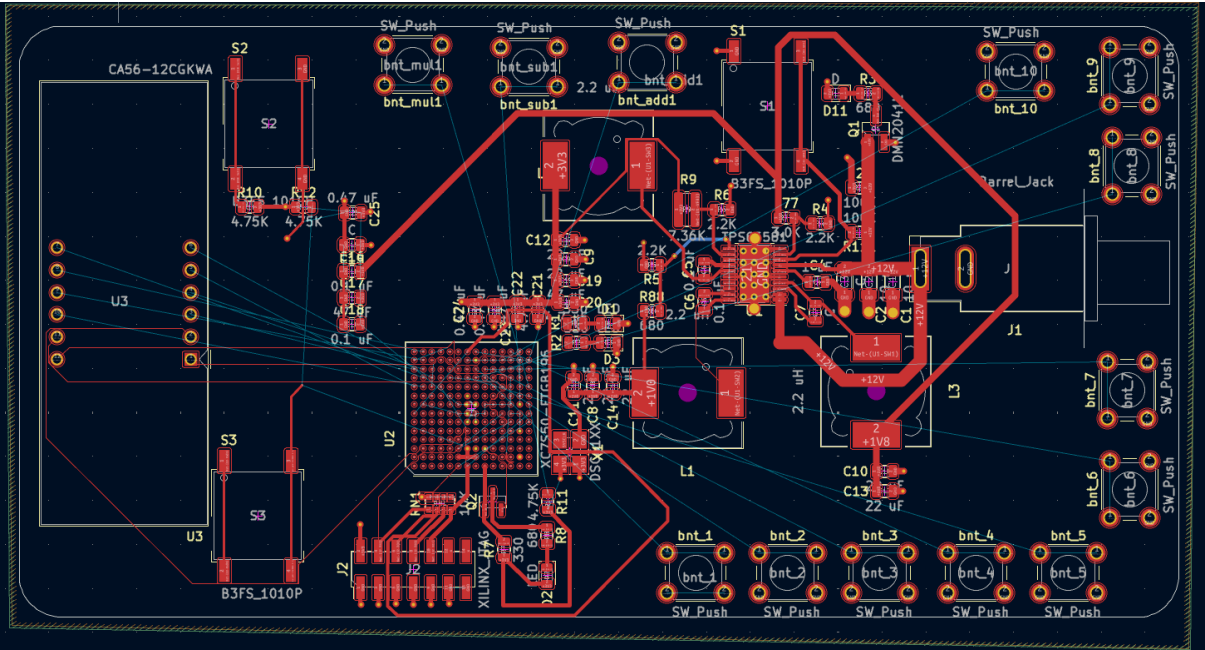


Fig 3: Board

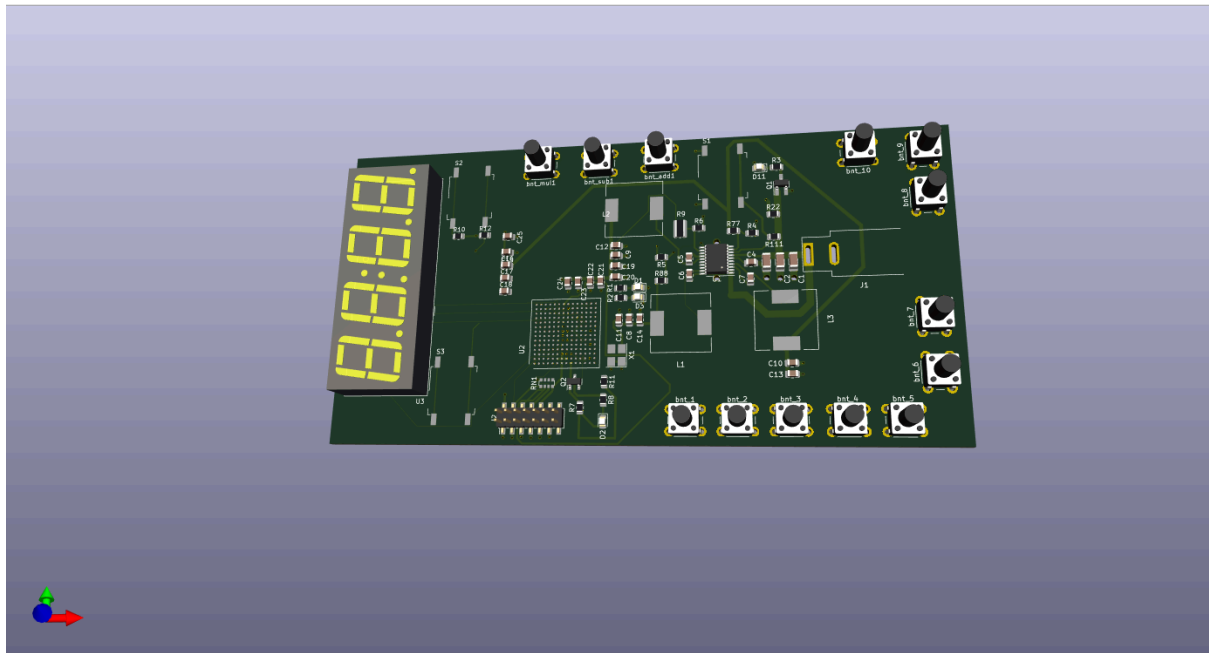


Fig 4: 3D view of the Board

8 Result

The calculator worked exactly as intended: every add or subtract operation on 4-bit inputs produced the correct decimal result on the seven-segment display, with no glitches or missed button presses. Timing reports showed it easily meets the 100 MHz clock, and it uses well under 1 % of the Artix-7's logic and BRAM resources. In short, the design is accurate, fast, and very compact—proof that simple HDL modules can deliver real-time arithmetic with minimal hardware.

9 References

- KiCad EDA. (n.d.). Learning resources. KiCad EDA.
<https://www.kicad.org/help/learning-resources/>
- Surf-VHDL. (2021, December 27). Vivado project tutorial - surf-VHDL.
<https://surf-vhdl.com/vivado-project-tutorial/>
- FPGA Learning
<https://www.xilinx.com/products/silicon-devices/resources/programming-an-fpga-an-introduction-to-how-it-works.html>
- Kicad sources <https://learn.sparkfun.com/tutorials/beginners-guide-to-kicad>